

Pricing Derivatives using Monte Carlo Methods

by

Nino Joly - Jules Rkalovic

EDHEC Business School and Université Côte d'Azur

Supervisor : **Mario Hernandez Tinoco**



June 2025

Table des matières

ABSTRACT	3
CHAPTER 1: TYPES OF OPTIONS.....	4
1.1 EUROPEAN OPTIONS	4
1.2 AMERICAN OPTIONS	5
1.3 BERMUDAN OPTIONS.....	6
1.4 ASIAN OPTIONS	6
1.5 BARRIER OPTIONS	7
1.6 WORST-OFF OPTIONS	9
1.7 BEST-OF OPTIONS.....	9
CHAPTER 2: PRICING METHODS.....	11
2.1 BLACK-SHOLES-MERTON MODEL.....	11
2.2 MONTE CARLO SIMULATIONS.....	13
2.2.1. Definition and usage	13
2.3.2 Deriving prices from stochastic processes	14
2.2.3 Numerical efficiency	14
2.2.4 Exotic options	16
CHAPTER 3: IMPROVEMENTS OF MONTE CARLO SIMULATIONS.....	19
3.1 VARIANCE REDUCTION METHODS	19
3.1.1 Antithetic Variates	19
3.1.2 Control Variates	21
3.1.3 Stratified Sampling.....	22
3.2. TESTING OF THEORETICAL FRAMEWORK	23
3.2.1 Antithetic Variates test	23
3.2.2 Control Variates	24
3.2.3 Stratified Sampling.....	25
CHAPTER 4: ALTERNATIVES AND EXTENSIONS.....	27
4.1 BINOMIAL TREES.....	27
4.2 LOCAL VOLATILITY MODELS.....	29
CONCLUSION	30
BIBLIOGRAPHY.....	31

ABSTRACT

This thesis investigates the use of Monte Carlo simulation techniques to price various financial derivatives. We begin by defining European, American, Bermudan, barrier and basket options, and then attach ourselves to explore the mathematical foundations for the Black Sholes model. Then, we introduce Monte Carlo methods as a flexible approach to compute the expected discounted payoff of an option under the risk-neutral measure. After that, we cover variance reduction techniques: antithetic variates, control variates, and stratified sampling. We explain each method's mathematical basis and show how they lower simulation error and accelerate convergence. To confirm the theoretical improvements, we apply them on real-world examples. Finally, we survey alternative numerical methods and extensions like binomial trees and local volatility models. We present the mathematical formulation of each approach and compare their suitability for different option types.

Keywords : Monte Carlo, pricing methods, derivatives, exotic options, Black Sholes, antithetic variates, control variates, stratified sampling, binomial trees, local volatility model, stochastic process

CHAPTER 1: TYPES OF OPTIONS

In today's financial markets, options have become indispensable tools for both risk management and speculation. They offer market participants the ability to hedge against adverse price movements, design structured investment strategies, or express targeted views on volatility and direction. Indeed, as they are traded OTC they are very flexible and with advanced pricing methods investors trust them more and more making them all the more attractive. Therefore, knowing more about them is essential to navigate financial markets and this is what we are going to do here.

To clarify the term, an option is a type of financial asset. It confers the right (but not the obligation) to buy (Call option) or sell (Put option) a specific quantity of an underlying asset or instrument at a specified strike price on or before a specified date, depending on the style of the option. In this thesis, we will only focus on stock options which are options based on an underlying stock.

1.1 EUROPEAN OPTIONS

European options are the simplest type of option. The buyer cannot exercise the right to buy or sell early. The payoffs are only tied to the stock price at the expiration date S_t and the strike price K .

-If the stock price at the expiration date is $S_t > K$:

The buyer of the Call can buy the underlying stock at the price K which is than the price of the stock S_t . Therefore, he will exercise his right to buy the stock at K and sell the stock at S_t . His profit (payoff) is $S_t - K > 0$.

-If the stock price at the expiration date is $S_t < K$:

The buyer of the Call has no advantage in buying the stock at K , as it is higher than the stock price at expiration. He will not exercise his right. Thus, the payoff is $0 > S_t - K$.

More generally, the payoff function of a long position for a European call option is:

$$\max(S_t - K; 0)$$

Similarly, we can compute the payoff functions of a short position for a European call option, and the equivalent for a European Put option.

We summarize the different functions in the following graph.



Figure 1: Payoffs of European Call and Put

1.2 AMERICAN OPTIONS

American options are similar to European options, but the difference is that the American options' holder can exercise their right on several dates $t \in [0, T] := \{0, \delta t, 2\delta t, 3\delta t, \dots, T\}$. At each date, we can compute the payoffs the same way as European options. For a long position for an American option, the payoff is given by the formula :

$$\varphi_{k \cdot \delta t} = \max(S_{k \cdot \delta t} - K, C')$$

where φ is the payoff function of the American call option and C' the payoff if we hold the options. We will consider that the possible expiration dates are all the dates between the emission of the option to its expiration. We cannot represent the payoff on a graph.

In Table 3, we provide the payoff functions for the different positions on American options.

$\forall k \in [0, T/\delta t]$	Long Position	Short Position
---------------------------------	---------------	----------------

American Call Option	$\varphi_{k \cdot \delta t} = \max(S_{k \cdot \delta t} - K, C')$	$\varphi_{k \cdot \delta t} = -\max(S_{k \cdot \delta t} - K, C')$
American Put Option	$\varphi_{k \cdot \delta t} = \max(K - S_{k \cdot \delta t}, C')$	$\varphi_{k \cdot \delta t} = -\max(K - S_{k \cdot \delta t}, C')$

Table 3 - Intermediate payoff functions for American Options

1.3 BERMUDAN OPTIONS

The Bermudan options is a mix of an American option and a European option. They have multiple exercising dates possible but, unlike the American options, it is only at given times.

Given E the set of all the possible expiration dates, in the table 3 there will be all the computations of the different payoff functions for the Bermudan options.

$\forall i \in E$	Long Position	Short Position
Bermudan Call Option	$\varphi_i = \max(S_i - K, 0)$	$\varphi_i = -\max(S_i - K, 0)$
Bermudan Put Option	$\varphi_i = \max(K - S_i, 0)$	$\varphi_i = -\max(K - S_i, 0)$

1.4 ASIAN OPTIONS

Asian options only have one expiration date and cannot be exercised early. But the difference with the European options is that the payoff is computed using the mean of the stock prices.

We will be considering the options using the mean of the stock prices at equally distanced dates through the time between the start of the contract and the expiration date with the arithmetic mean (there is no need to use the Monte Carlo Methods with the geometric mean Asian options as closed forms exist).

In the table 4, we will resume the payoff functions for a long position an Asian options :

	Long Position	Short Position
Asian Call Option	$\max(\bar{Y} - K, 0)$	$-\max(\bar{Y} - K, 0)$
Asian Put Option	$\max(K - \bar{Y}, 0)$	$-\max(K - \bar{Y}, 0)$

Table 4 - Payoff functions for Arithmetic Asian Options where $\bar{Y} = (1/T) \cdot \sum S_i$ with S_i the underlying price at time i .

Asian options are popular in commodities, currency and equity-linked products because averaging reduces price volatility and end-point risk. By smoothing the payoff, an Asian option can mitigate market-manipulation risk near expiry and lowers implied volatility (and thus premium) relative to a vanilla option. For example, Asian structures are used in equity-linked notes and annuities to smooth out returns on the equity component. Commodity traders may prefer an Asian option to hedge an average price over a delivery period rather than a spike at maturity. They are also used in corporate finance. Indeed, Asian calls were introduced to meet reporting rules because their lower cost (due to averaging) made them cheaper to expense than vanilla options. In structured products, embedding an Asian payoff helps reduce upside volatility while still offering participation: studies show that an embedded Asian option “smoothes” the distribution of payoffs and can improve the breakeven probability of a product by lowering risk.

1.5 BARRIER OPTIONS

A barrier option is an option where the payoff depends on whether the underlying asset's price goes under or over a predefined level. The most common ones are :

-Knock-out options : the option expires automatically when the stock's price touches one or several predefined levels.

-Knock-in options : the option is activated only if the levels are touched.

These barrier options are structured like Call and Put. For example:

-A DOC (down-and-out Call) with a strike K , a barrier H and a maturity of T is the possibility to buy the stock at K at the time T only if the stock's price remained over H .

-A UOC (up-and-out Call) is the same idea but if the stock price remained under H .

-A DIC (down-and-in Call) is activated if the stock price goes under H

-A UIC (up-and-in Call) is activated if the stock price goes above H

The same definitions apply to the for the Put.

In the table 6, there are all the different payoffs for these barrier options.

	Long Position	Short Position
UO Call Option	$\max(S_T - K, 0) \cdot 1_{\{\max_t S(t) < B\}}$	$-\max(S_T - K, 0) \cdot 1_{\{\max_t S(t) < B\}}$

UO Put Option	$\max(K - S_T, 0) \cdot 1_{\{\max_t S(t) < B\}}$	$-\max(K - S_T, 0) \cdot 1_{\{\max_t S(t) < B\}}$
UI Call Option	$\max(S_T - K, 0) \cdot 1_{\{\max_t S(t) \geq B\}}$	$-\max(S_T - K, 0) \cdot 1_{\{\max_t S(t) \geq B\}}$
UI Put Option	$\max(K - S_T, 0) \cdot 1_{\{\max_t S(t) \geq B\}}$	$-\max(K - S_T, 0) \cdot 1_{\{\max_t S(t) \geq B\}}$
DO Call Option	$\max(S_T - K, 0) \cdot 1_{\{\min_t S(t) > B\}}$	$-\max(S_T - K, 0) \cdot 1_{\{\min_t S(t) > B\}}$
DO Put Option	$\max(K - S_T, 0) \cdot 1_{\{\min_t S(t) > B\}}$	$-\max(K - S_T, 0) \cdot 1_{\{\min_t S(t) > B\}}$
DI Call Option	$\max(S_T - K, 0) \cdot 1_{\{\min_t S(t) \leq B\}}$	$-\max(S_T - K, 0) \cdot 1_{\{\min_t S(t) \leq B\}}$
DI Put Option	$\max(K - S_T, 0) \cdot 1_{\{\min_t S(t) \leq B\}}$	$-\max(K - S_T, 0) \cdot 1_{\{\min_t S(t) \leq B\}}$

Table 6 - Payoff functions for Barrier Options

Barrier options can be seen as conditioned vanillas, and typically carry a lower premium than a plain option with the same strike, because one of the payoff scenarios is removed. For the buyer, this cost saving can be used to boost yield or provide other features. For example, an issuer might sell an up-and-in call (cheaper than a regular call) to provide upside if the market rises, knowing it won't have to pay unless the barrier is hit. In structured products, barriers let issuers offer fixed coupons with protection buffers. A common example is a barrier reverse convertible essentially a short barrier-put which pays a high coupon if the underlying never drops below a barrier, acting like partial principal protection. Barrier options' lower premiums make them affordable for investors wishing to protect their capital and allow investors to customize strike and barrier to their outlook.

However, barriers introduce path dependency and discontinuities, making them harder to price and its sensitivities (Greeks) large. Once a barrier is touched, payoffs can drop to zero abruptly. Moreover, real markets have fat tails: large moves can trigger knock-out more often than Black–Scholes would predict, and out-barriers are susceptible to market manipulation if prices can be pushed just over the barrier. Thus, although barriers reduce upfront cost, they add execution and model risk: unanticipated barrier events can cause total loss. In summary, barrier options offer structured payoff flexibility at reduced cost, but at the price of increased complexity and path-risk. They are a double-edged tool, very useful for customizing structured notes, yet requiring careful pricing (and this is where Monte Carlo simulations will be able to express their potential).

1.6 WORST-OF OPTIONS

A worst-of option on a basket of n assets pays based on the worst-performing (smallest) asset in the basket. Concretely, if $S_i(T)$ are the asset prices at maturity, define $S_{\min}(T) = \min_i S_i(T)$. A worst-of call's payoff is

$$\max(S_{\min}(T) - K, 0),$$

while a worst-of put pays

$$\max(K - S_{\min}(T), 0)$$

Worst-of options emphasize downside scenarios. For example, a worst-of call is relatively cheap (since it depends on the minimum asset) and is used by an investor who wants leveraged exposure to the least performing asset in the basket. Conversely, a worst-of put has a higher payoff potential than a vanilla basket put, because if any asset plummets, the option pays off heavily. Structured products often use worst-of puts: a common design is that an investor buys a bond and sells a worst-of put on several stocks. In such reverse-convertible structures, the worst-of put may include barrier (knock-in) features, but essentially the seller collects a large premium. Intuitively, selling a worst-of put yields more income (since many outcomes trigger payoff) but carries the risk that one asset's crash will force a large payoff.

Worst-of options are especially sensitive to correlation. If the assets are highly correlated, they tend to move together, so the difference between the minimum and other assets is small, making the worst-of call more valuable. In contrast, if correlation is low (high dispersion), it is more likely one asset underperforms the others, which increases the expected payoff of a worst-of put. Overall, buyers of both best-of calls and worst-of puts are short correlation: they prefer lower correlation between assets as it raises the chance of an extreme outcome in the basket. (In plain terms, diversification hurts a worst-of put seller because “anyone failing” is more probable when assets move independently.)

1.7 BEST-OF OPTIONS

A best-of option (also called an “either-or” or “rainbow” option) pays based on the best-performing asset in a basket. If $S_{\max}(T) = \max_i S_i(T)$, then a best-of call has payoff

$$\max(S_{\max}(T) - K, 0),$$

and a best-of put pays

$$\max(K - S_{\max}(T), 0).$$

Intuitively, the best-of call lets the holder benefit from whichever asset does best: the payoff is as if one had chosen the top-performing underlying. Best-of options are the opposite tilt of a worst-of: a best-of call is more expensive than a vanilla basket call (since it uses the maximum) and its holder is effectively long correlation. Best-of puts are cheaper than comparable vanilla puts, because they pay only if all assets fall enough that even the best one is below strike.

CHAPTER 2: PRICING METHODS

As we mentioned before, in the last 50 years options have gained a lot of attention and became widely used. To make sure the prices were set correctly and to profit from potential arbitrage, a whole field of financial mathematics was developed, leading for example to advances in stochastic processes. Here we will explore how it all began and the particular case of Monte Carlo simulations which are still widely used today.

2.1 BLACK-SHOLES-MERTON MODEL

In the 1970s, Fisher Black, Myron Sholes and Robert C. Merton developed a mathematical model which linked the implicit price of an option with the change in price of the underlying asset. In order to follow this model, there are assumptions that need to be made:

- The price S follows a Geometric Brownian Motion (GBM) which means that $dS = \mu S dt + \sigma S dz$ with μ the expected return and σ the volatility and dz a Wiener process
- Time is continuous
- There is no arbitrage opportunity
- Short selling is allowed
- The risk-free rate of interest is constant and same for all maturities
- There is no transaction costs
- All securities can be divided
- No dividends paid between the pricing of the option and the maturity

The GBM assumption leads to interesting properties. Indeed, it is a continuous process and with it S remains positive which fits for a stock price. After that, we can use Itô's lemma to produce a discretized version of this GBM which will be exactly true for any interval.

For a stochastic process $dx = a dt + b dz$ and a function $G(x,t)$, Itô's lemma tells us that $G(x)$ follows the process:

$$dG = \left(\frac{\partial G}{\partial x} \mu S + \frac{\partial G}{\partial t} + 0.5 \frac{\partial^2 G}{\partial x^2} \sigma^2 S^2 \right) dt + \frac{\partial G}{\partial x} \sigma S dz$$

(2.1)

Thus when we apply the lemma it gives :

$$df = \left(\frac{\partial f}{\partial S} \mu S + \frac{\partial f}{\partial t} + 0.5 \frac{\partial^2 f}{\partial S^2} \sigma^2 S^2 \right) dt + \frac{\partial f}{\partial S} \sigma S dz$$

To get to the Black-Sholes-Merton partial differential equation, let's consider a portfolio Π consisting in 1 short position in the derivative and n long position in the stock ($\Pi = -f + nS$). So the process is $d\Pi = -df + ndS$ and another way to express it is with the expression of df and dS :

$$d\Pi = -\left[\left(\frac{\partial f}{\partial S} - n \right) \mu S + \frac{\partial f}{\partial t} + 0.5 \frac{\partial^2 f}{\partial S^2} \sigma^2 S^2 \right] dt - \left(\frac{\partial f}{\partial S} - n \right) \sigma S dz$$

To get rid of the stochastic part, we take $n = \frac{\partial f}{\partial S}$.

Through these computations, we have managed to get a riskless portfolio Π , which should therefore earn the risk free rate : $d\Pi = \Pi r dt = \left(-f + \frac{\partial f}{\partial S} S \right) r dt$

This ends the demonstration as we get the partial differential equation we were looking for :

$$\frac{\partial f}{\partial t} + rS \frac{\partial f}{\partial S} + 0.5 \frac{\partial^2 f}{\partial S^2} \sigma^2 S^2 = rf \quad (2.2)$$

To solve this requires boundary conditions, which are given by the payoffs of the option studied.

For instance, if we take a European call on a stock that pays no dividend, the payoff function is given by $f(S, T) = \max(S_T - K, 0)$ where S_T is the spot price at maturity and K is the strike price. The Black Sholes formula in that case gives the price c of the option :

$$c = S * N(d_1) - K * e^{-rt} * N(d_2) \quad (2.3)$$

where $d_1 = \frac{\ln\left(\frac{S}{K}\right) + \left(r + \frac{1}{2}\sigma^2\right)T}{\sigma\sqrt{T}}$ and $d_2 = d_1 - \sigma\sqrt{T}$

Here $N(\cdot)$ stands for the standard normal cumulative distribution function and can therefore be found on tables.

If we want to find the price of a European put option, a simple way to do it is to use the put-call parity which again relies on risk neutral valuation:

$$p = c - S + K * e^{-rt} \quad (2.4)$$

Ex: From the website barchart.com, we can have a look at the options traded on a stock, their price and the parameters associated with it. Here, to test Black Sholes method with the empirical truth, we have chosen a European call on AAPL expiring on 06/02/2025 with 4 days left to maturity, a spot price $S=201.7$, a strike price $K=200$, $T=4/365$, volatility=32.02% and a risk-free interest rate = 4.12%.

All these parameters applied to the adapted Black Sholes formula for a European call give us $c= 3.67$, which is the price at which this option was trading at the time. We thus observe that Black Sholes gives us the correct price for a simple option.

As we have seen, Black Sholes relies heavily on boundary conditions to solve the PDE and struggles to give closed form equations for more exotic or path dependent options. In that case, a way to price options is using Monte Carlo methods.

2.2 MONTE CARLO SIMULATIONS

2.2.1. Definition and usage

First, let's define what Monte Carlo simulations are to better understand why they are useful in our situation and how we are going to use them.

Monte Carlo simulations are a class of computational algorithms that rely on random sampling to obtain numerical results. They are widely used when dealing with systems that exhibit uncertainty and complex dynamics, particularly when analytical solutions are difficult or impossible. Their usage goes from measuring uncertainties on physical measures to simulating stock trajectories.

The idea, based on the Law of Large Numbers, is simple: simulate many possible future paths of the underlying asset(s), calculate the desired quantity (here the payoff), and average the outcomes under the appropriate measure (here risk-neutral).

Th : (**Law of Large Numbers**) Let $X_1, X_2, \dots, X_n$ be independent and identically distributed (i.i.d.) random variables with finite expected value $\mu=E[X_i]$. Then:

$$\frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{P} \mu \text{ as } n \rightarrow \infty$$

In our case, we will therefore calculate the present value of the derivatives' expiration payoff for each price trajectory and then take the mean of this to find an estimate for the price of the option. The more trajectories we take into account, the better the estimate will be but this requires time and resources in a context where time is of the essence.

2.3.2 Deriving prices from stochastic processes

In part 2.2. we assumed that the stock price S follows a Geometric Brownian Motion and derived an equation from this using Itô's lemma. We are going to use equation (2.1) on $G=\ln(S)$ when S follows a GBM :

$$dS = \mu S dt + \sigma S dz$$

Having :

$$\frac{\partial G}{\partial t} = 0; \frac{\partial G}{\partial S} = \frac{1}{S}; \frac{\partial^2 G}{\partial S^2} = -\frac{1}{S^2}$$

This gives :

$$d\ln(S) = \left(\mu - \frac{\sigma^2}{2}\right)dt + \sigma dz$$

But another thing we know is that when $\delta t \rightarrow 0$, $d\ln(S) = \ln(S_{t+\delta t}) - \ln(S_t) = \ln(S_{t+\delta t}/S_t)$ so exponentiating both sides we get :

$$S_{t+\delta t} = S_t * e^{(\mu - \frac{\sigma^2}{2})dt + \sigma \varepsilon \sqrt{\delta t}} \quad (2.5)$$

Where ε is a standard normal draw (and noticing that $dz = \varepsilon \sqrt{\delta t}$)

The expression (2.5) gives us all the information we need to conduct the Monte Carlo simulations explained above.

Ex : In the case of a European call, there is only one period so a possible price is given by $S_T = S_0 * e^{(\mu - \frac{\sigma^2}{2})T + \sigma \varepsilon \sqrt{T}}$

2.2.3 Numerical efficiency

When pricing options, being precise and reducing uncertainty are key elements to take into consideration. To do so, we often use a key statistic tool : the central limit theorem :

Th: (Central Limit Theorem) Let $X_1, X_2, \dots, X_n$ be independent and identically distributed (i.i.d.) random variables with finite expected value $\mu = E[X_i]$ and finite variance $\sigma^2 = \text{Var}(X_i)$.

The sample mean is : $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$

Then as n converges toward the infinity :

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0,1)$$

This key result allows us to build confidence intervals, which can be useful when the price needs to reach certain level of precision.

Ex : Thanks to the CLT, for an estimate $\bar{f}_n$, we can choose a half width for which we are 95% confident that the true value is within the chosen interval by picking $\text{Halfwidth} = 1.96 \frac{s}{\sqrt{n}}$ where 1.96 is the z-score for a 95% confidence interval and s is the standard deviation of the discounted payoffs (unknown in advance).

Suppose that, for pricing purposes, we want a half width of 0.005. Then a way to get the number of computations required could be to compute s empirically for a small number of iterations, solve the equation and then repeat the Monte Carlo simulation the corresponding number of times necessary to match the width desired (see appendix for the code).

In the case of the AAPL European call we study, results show that over 800k simulations are required to get the level of precision we want, which we set to 0.005 for the half-width.

Then, another important thing to see is how fast this probabilistic method is at finding option prices when opposed to the basis of the financial system for vanilla options, the Black Sholes equation.

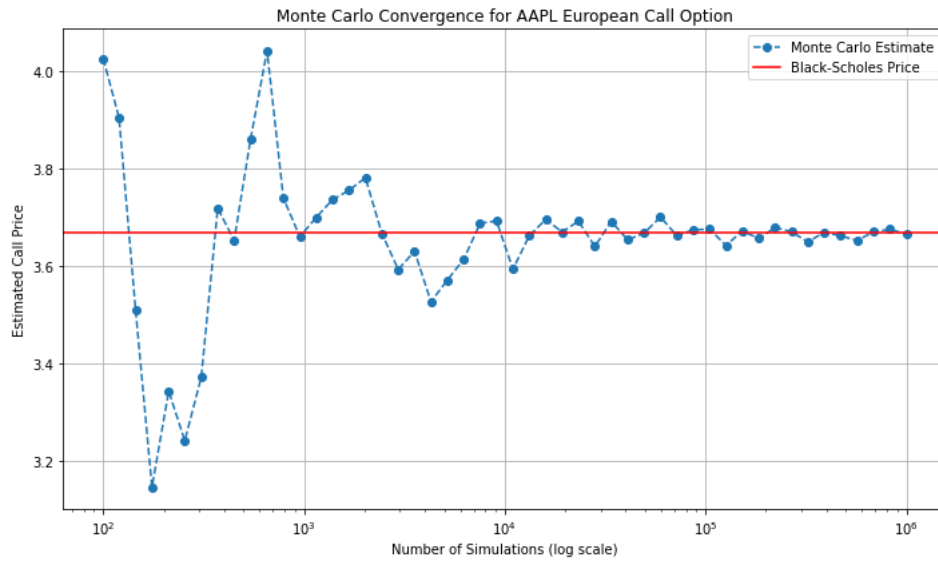


Figure 2: Convergence speed of MC simulations

The empirical convergence shows that it takes approximately 1 million simulations to get the Black Sholes price, which we got in just a few operations. To quantify this convergence to the correct result, we can see that Black Sholes has a complexity of $O(1)$ while Monte Carlo simulations converge at the rate $O(1/\sqrt{N})$, N being the number of simulations. This shows that for options where closed form solutions exist, Black Sholes is king, and Monte Carlo simulations are just a tool to experience with. However, when facing exotic/ path dependent options, Monte Carlo simulations have an edge, and this is what we are going to see.

2.2.4 Exotic options

So far, we have used Monte Carlo simulations on a European call and observed the result. Now, it is time to see how these methods behave for Asian and barrier options as well as worst off options.

1) Asian option:

When applying our algorithm (see appendix for further explications), we find a price of 2.56 which is lower than the European call price. This comes from the choice of parameters in our example. Indeed, here we took a maturity of 4 days, which is very short and leaves little room for the price to change, especially considering an Asian option where changes are even lower as they are averaged over the period.

2) Barrier Down-and-Out call option:

For this option, the parameters we took are the same as for the other options and we focused only on changing the level of the barrier to see what happens, the results are shown in the graph below. To price it, the concept is to do the same thing as for the Asian option, meaning we calculate the path of the option, and if the barrier has been breached then the payoff for the simulation is set to zero.

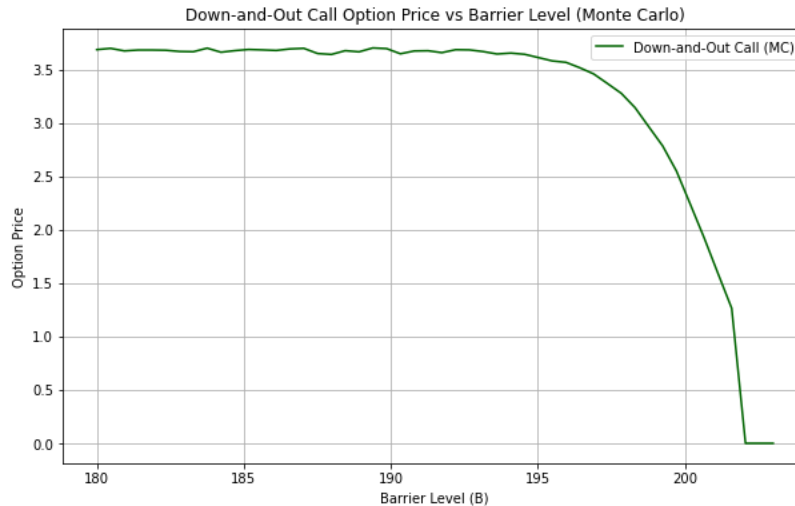


Figure 3: Change in price of a barrier option as the barrier level moves

A way to understand the price shown above is the following: if the barrier is very low, then the stock price has almost no chance of ever hitting the barrier and knocking back the option. This shows why for low barrier levels the price is almost the same as for a vanilla European call. The higher the barrier gets, the more likely the option is to be knocked back so the price goes down accordingly. Finally, for barrier levels superior to the spot price the option would be knocked back immediately so its virtual value would be 0.

3) Worst-Of:

To price this, we are again going to assume that the price of the stocks follows a GBM but we need to take into account that the stocks might be correlated between them. If that wasn't the case then the pricing would be more straightforward. Instead, we are going to use a matrix of correlation Σ and a gaussian vector $Z = (Z_1, Z_2, \dots, Z_n) \sim \mathcal{N}(0, \Sigma)$.

In order to simulate correlated standard normal, we use $\varepsilon \sim \mathcal{N}(0, I_n)$, the Cholesky decomposition and find L such that $\Sigma = LL^T$ to express $Z = L * \varepsilon$.

For this worst-of basket option, we are going to take 3 stocks (AAPL, NVDA, TSLA) and take an interest in their performance rather than spot price, so that we can compare them more easily at the end.

This worst-of, calculated on the 2nd of June 2025 will have an expiration date on the 6th June 2025. By that time, NVDA had an implied vol of 36% and was quoted at 135.50\$. For TSLA, the IV was at 64% and quoted at 343.83\$.

The correlation between the stock prices is hard to find online, we thus set up a time period of one year between June 2024 and June 2025 to gather the stock prices from Yahoo Finance where we computed the Pearson correlation between all pairs, which were then used to fill the correlation matrix Σ :

Correlation	AAPL	NVDA	TSLA
AAPL	1	0.44	0.50
NVDA	0.44	1	0.50
TSLA	0.50	0.50	1

These are the different results we get when varying the strike price K:

Percentage K	Option price (in % of the notional)
K=0%	0.5
K=-5%	2.9
K=-10%	7.14
K=5%	0

CHAPTER 3: IMPROVEMENTS OF MONTE CARLO SIMULATIONS

3.1 VARIANCE REDUCTION METHODS

Monte Carlo simulations require a lot of iterations to perform accurately, due to the nature of their convergence explained and highlighted in the previous chapter. To face this problem, numerous techniques have been developed and we are going to explore some of them.

3.1.1 Antithetic Variates

Theorem 3.1: (Negative covariance of antithetic samples): Let $X = h(U_1, \dots, U_k)$ be a random variable generated by a function h that is monotone (either nondecreasing or nonincreasing) in each argument, with $U_1, \dots, U_k$ independent Uniform(0,1). Define its antithetic counterpart $X' = h(1 - U_1, \dots, 1 - U_k)$. Then

$$\text{Cov}(X, X') \leq 0,$$

i.e. X and X' are negatively correlated.

Proof: Since $(1 - U_1, \dots, 1 - U_k)$ has the same joint distribution as $(U_1, \dots, U_k)$, it follows that X and X' have the same marginal distribution and mean. Monotonicity of h in each argument ensures the rearrangement inequality (or simple coupling argument) applies as each U_i increases, $1 - U_i$ decreases, so $h(U)$ and $h(1 - U)$ move in opposite directions. Formal proofs (see e.g. Glasserman) show this implies $\text{Cov}(X, X') \leq 0$ $\square$

Theorem 3.2 (Variance reduction by antithetic averaging): Let X and X' be identically distributed with common variance $\text{Var}(X) = \sigma^2$ and covariance $\text{Cov}(X, X') = \gamma \leq 0$. Define the antithetic average $Y = (X + X')/2$. Then

$$\text{Var}(Y) = \frac{\text{Var}(X) + \text{Cov}(X, X')}{2} = \frac{\sigma^2 + \gamma}{2} \leq \frac{\sigma^2}{2} < \sigma^2$$

In particular, $\text{Var}(Y) \leq \text{Var}(X)$, with strict inequality when $\text{Cov}(X, X') < 0$.

Proof: By properties of variance,

$$\text{Var}(Y) = \text{Var}\left(\frac{X + X'}{2}\right) = \frac{1}{4}(\text{Var}(X) + \text{Var}(X') + 2\text{Cov}(X, X'))$$

Since X' has the same distribution as X , $\text{Var}(X') = \text{Var}(X) = \sigma^2$. Hence

$$\text{Var}(\widehat{\mu}_A) = \frac{1}{2n}(2\sigma^2 + 2\gamma) = \frac{\sigma^2 + \gamma}{n}$$

If $\gamma < 0$, then $\text{Var}(Y) < \sigma^2/2 < \sigma^2$, proving variance reduction. Even if $\gamma = 0$, $\text{Var}(Y) = \sigma^2/2 < \sigma^2$; equality $\text{Var}(Y) = \text{Var}(X)$ only in the degenerate case $\gamma = \sigma^2 = 0$.

By repeatedly pairing independent samples into antithetic pairs and averaging as above, the resulting Monte Carlo estimator has smaller variance than naïve averaging. In practice one generates $(U_1, \dots, U_k)$ and its antithetic $(1 - U_1, \dots, 1 - U_k)$ for each sample, computing payoffs X and X' , and then uses their average $Y = (X + X')/2$ as a new unbiased sample. Because of negative covariance, the sample variance of Y is reduced compared to that of X .

We can apply this general solution to Gaussian estimates. This method exploits symmetric negative dependence between paired paths generated using complementary normal variates. For each path simulated with $Z_1, Z_2, \dots, Z_d$, a mirror path is generated with $-Z_1, -Z_2, \dots, -Z_d$.

The estimator becomes:

$$\widehat{\theta}_{AV} = \frac{1}{2N} \sum_{i=1}^N [f(S_T^{(i+)}) + f(S_T^{(i-)})]$$

where $S_T^{(i+)}$ and $S_T^{(i-)}$ denote the terminal prices from positive and negative variates. The variance reduction depends on:

$$\text{Var}(\widehat{\theta}_{AV}) = \frac{\sigma^2}{N} (1 + \rho)$$

where $\rho = \text{corr}\left(f\left(S_T^{(i+)}\right), f\left(S_T^{(i-)}\right)\right) < 0$. The generalized K-antithetic estimator extends this concept using K negatively correlated normal vectors.

3.1.2 Control Variates

Consider an estimator X (with unknown mean $\mu = E[X]$) and a control variate Y for which the expectation $E[Y] = \nu$ is known and $\text{Cov}(X, Y) \neq 0$. A general control-variate estimator is

$$\widehat{\mu}_\beta = \bar{X} - \beta(\bar{Y} - \mu_Y) \quad \text{with} \quad \bar{X} = \frac{1}{n} \sum_{i=1}^n X_i, \quad \bar{Y} = \frac{1}{n} \sum_{i=1}^n Y_i$$

This estimator is unbiased for any β (since $E[Y_i] = \nu$). Its variance is

$$\text{Var}(\widehat{\mu}_\beta) = \frac{1}{n} (\text{Var}(X) + \beta^2 \text{Var}(Y) - 2\beta \text{Cov}(X, Y))$$

Minimizing this quadratic in β yields the optimal coefficient.

Theorem 3.3 (Optimal control coefficient): The variance of the control-variate estimator is minimized by

$$\beta^* = \frac{\text{Cov}(X, Y)}{\text{Var}(Y)}$$

Substituting β^* gives

$$\text{Var}(\widehat{\mu}_{\beta^*}) = \frac{\text{Var}(X)}{n} (1 - \rho_{XY}^2)$$

where $\rho_{XY} = \text{Cov}(X, Y) / \sqrt{\text{Var}(X)\text{Var}(Y)}$ is the correlation coefficient.

Proof: Setting the derivative of $\text{Var}(X - \beta Y)$ w.r.t. β to zero yields $\beta^* = \text{Cov}(X, Y) / \text{Var}(Y)$. Plugging this into the variance formula:

Since $0 \leq \rho_{XY}^2 \leq 1$, it follows $\text{Var}(\widehat{\mu}_{\beta^*}) \leq \text{Var}(X)/n$. $\square$

Corollary 3.4: Because $1 - \rho_{XY}^2 \leq 1$, the variance of the optimally controlled estimator is never larger than the original estimator (it is strictly smaller whenever $|\rho_{XY}| > 0$). In particular,

$$\text{Var}(\widehat{\mu}_{\beta^*}) = \frac{\text{Var}(X)}{n} (1 - \rho_{XY}^2) \leq \frac{\text{Var}(X)}{n}.$$

Equality holds only if X and Y are uncorrelated ($\rho_{XY} = 0$). This variance decomposition highlights that the greater the linear correlation $|\rho_{XY}|$ between X and Y , the larger the reduction.

3.1.3 Stratified Sampling

Suppose the population or input domain is partitioned into K disjoint strata with (known) probabilities $p_1, \dots, p_K$. In each stratum k , we draw n_k independent samples of the random variable Y (e.g. the payoff) conditional on being in stratum k . Let $\widehat{\mu}_k$ be the unbiased estimator of $E[Y \mid \text{stratum } k]$ from those samples. The stratified estimator of $\mu = E[Y]$ is

$$\widehat{\mu}_S = \sum_{k=1}^K p_k \widehat{\mu}_k$$

Theorem 3.5 (Unbiasedness): The stratified estimator is unbiased: $E[\widehat{\theta}_{\text{strat}}] = \theta$.

Indeed, by the law of total expectation

$$\theta = E[Y] = \sum_{k=1}^K p_k E[Y \mid \text{stratum } k] = \sum_{k=1}^K p_k \theta_k$$

so $E[\widehat{\theta}_k] = \theta_k$ implies $E[\widehat{\theta}_{\text{strat}}] = \sum p_k \theta_k = \theta$. $\square$

Theorem 3.6 (Variance): If $\sigma_k^2 = \text{Var}(Y \mid \text{stratum } k)$ and the strata samples are independent, then

$$\text{Var}(\widehat{\mu}_S) = \sum_{k=1}^K \frac{p_k^2}{n_k} \sigma_k^2$$

In particular, under proportional allocation $n_k = np_k$, this becomes

$$n_k = Np_k \quad \Rightarrow \quad \text{Var}(\widehat{\mu}_S) = \frac{1}{N} \sum_{k=1}^K p_k \sigma_k^2$$

Compare this to the usual Monte Carlo variance $\text{Var}(\bar{Y}) = \sigma^2/n$ with $\sigma^2 = \text{Var}(Y) = \sum p_k \sigma_k^2 + \text{Var}(\theta_k)$ by the law of total variance. From the second term $\text{Var}(\theta_k) \geq 0$, one sees immediately $\sum p_k \sigma_k^2 \leq \sigma^2$. Hence

$$\text{Var}(\widehat{\mu}_S) = \sum_{k=1}^K \frac{p_k^2}{n_k} \sigma_k^2 \leq \frac{\sigma^2}{N} = \text{Var}(\bar{Y})$$

with equality only if $\text{Var}(\theta_k) = 0$ (all stratum means equal). Thus, stratification cannot increase variance, and typically reduces it unless the strata are ill-chosen.

Proof: The variance formula follows by noting each $\widehat{\theta}_k$ has variance σ_k^2/n_k and the weighted sum has variance $\sum p_k^2 \sigma_k^2/n_k$. Under proportional allocation $n_k = np_k$, this simplifies as above. The law of total variance yields $\sigma^2 = \sum p_k \sigma_k^2 + \sum p_k (\theta_k - \theta)^2 \geq \sum p_k \sigma_k^2$, giving the inequality. $\square$

In practice one often stratifies along one or more key dimensions. In one-dimensional stratification (e.g. subdividing $[0,1]$ into intervals of lengths p_k), the above formulas apply directly. In high-dimensional settings, a full grid stratification would require exponentially many strata. However, for very high dimension the gain from stratification diminishes, as it can substantially improve convergence for smooth low-dimensional integrals but has little effect in high dimensions.

3.2. TESTING OF THEORETICAL FRAMEWORK

To test if these methods are more effective than the classical Monte Carlo method, we made graphs using the same parameters we did on the Apple European Call. These graphs focus on the comparison of the estimated option's price for 2 different methods depending on the number of simulations.

3.2.1 Antithetic Variates test

The antithetic method (green line) demonstrates notably smoother convergence toward the limit price of approximately 3.67, while the classical Monte Carlo method (blue dashed line) exhibits more erratic oscillations throughout the simulation range.

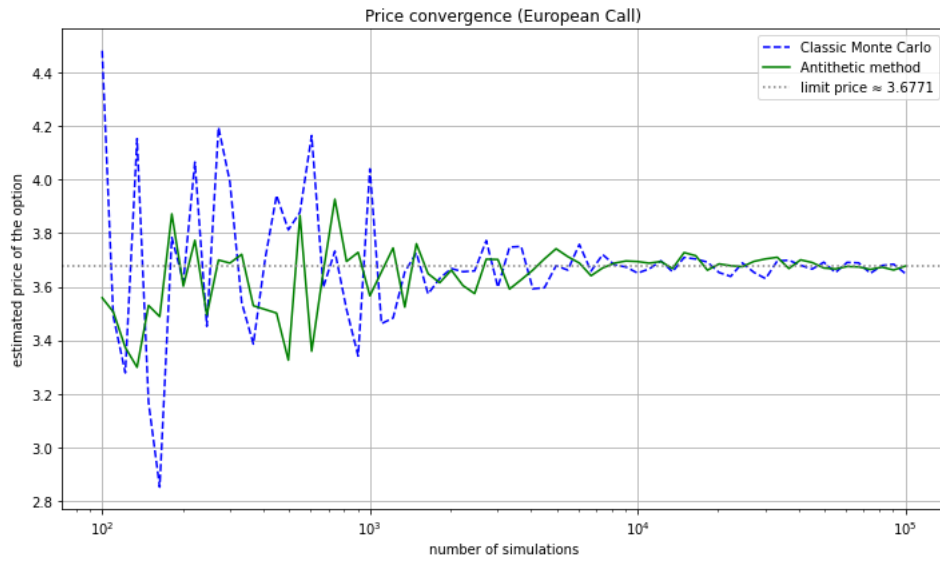


Figure 4: Comparison between Antithetic variates and MC simulations

The antithetic variates method achieves this superior convergence through its fundamental principle of generating negatively correlated pairs of random variables. For each uniform random variable U generated in the simulation, the method simultaneously uses its antithetic counterpart $(1-U)$, which maintains the same distribution but introduces negative correlation. This negative correlation effectively reduces the variance of the estimator by balancing extreme outcomes, when one path produces an unusually high value, its antithetic counterpart tends to produce a correspondingly low value.

A more accurate, less visual way to compare these is through the Efficiency Ratio which is $Efficiency\ Ratio = \frac{\sigma_{MC}^2}{\sigma_{AV}^2}$. Here, when we compute it we find an efficiency ratio of 5.24 which means that the antithetic variates require 5.24 fewer simulations to reach the same level of precision.

3.2.2 Control Variates

The graph we get for the control variates is fairly similar to the graph above, showing better results than traditional Monte Carlo simulations.

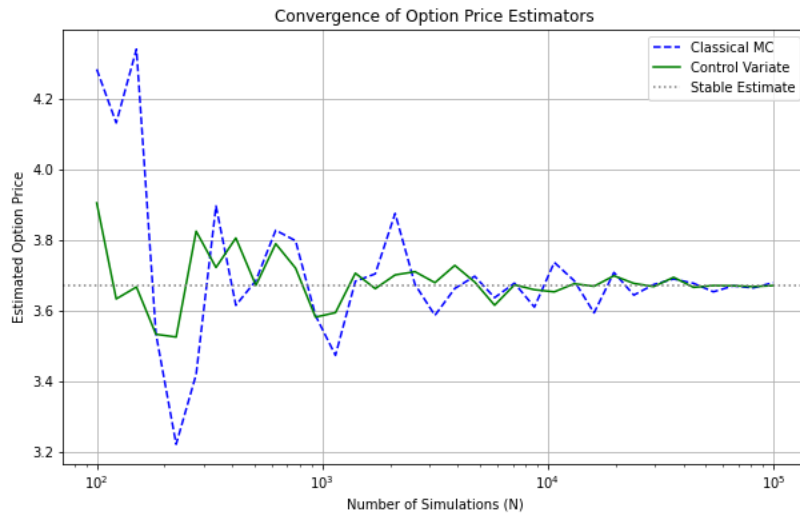


Figure 5: Comparison of convergence speed between Control Variates and MC simulations

The graph below provides compelling evidence of the control variates method's computational superiority. Indeed, the logarithmic scale reveals that control variates consistently achieve lower standard errors compared to classical Monte Carlo across all sample sizes, a result again confirmed by the computation of the Efficiency Ratio which reaches 5.49 for this method.

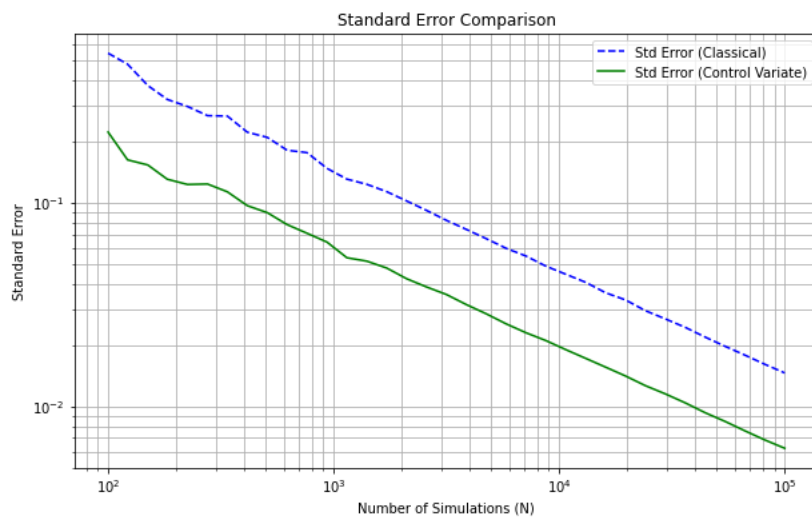


Figure 6: Comparison of standard errors between Control Variates and MC simulations

3.2.3 Stratified Sampling

For this method, stratified sampling runs very efficiently and reaches the price much earlier than Monte Carlo simulations.

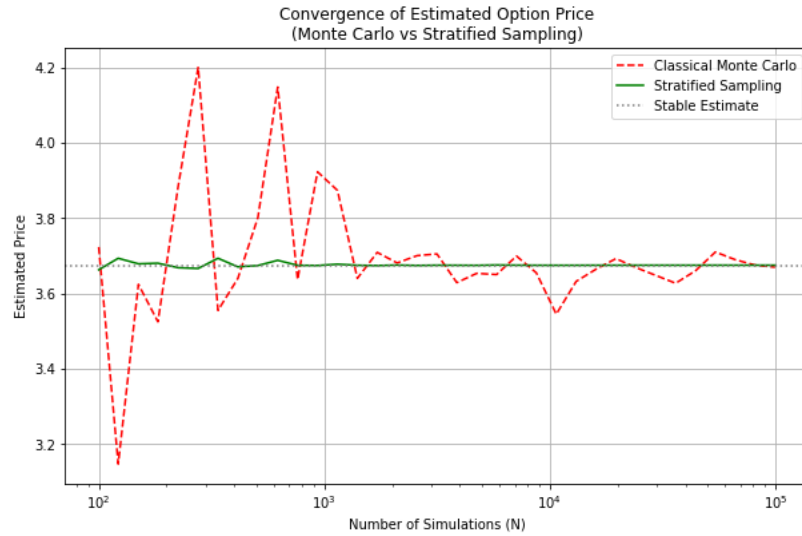


Figure 7: Comparison of convergence speed between Stratified Sampling and MC simulations

However, the option we are pricing here is a European call which is not highly non-linear (as opposed to path dependent options for instance) and the strata are uniform and not especially focused on the strike price. This explains why as the number of simulations grows, the efficiency ratio diminishes until it reaches 1, thus providing the same results as a MC simulation:

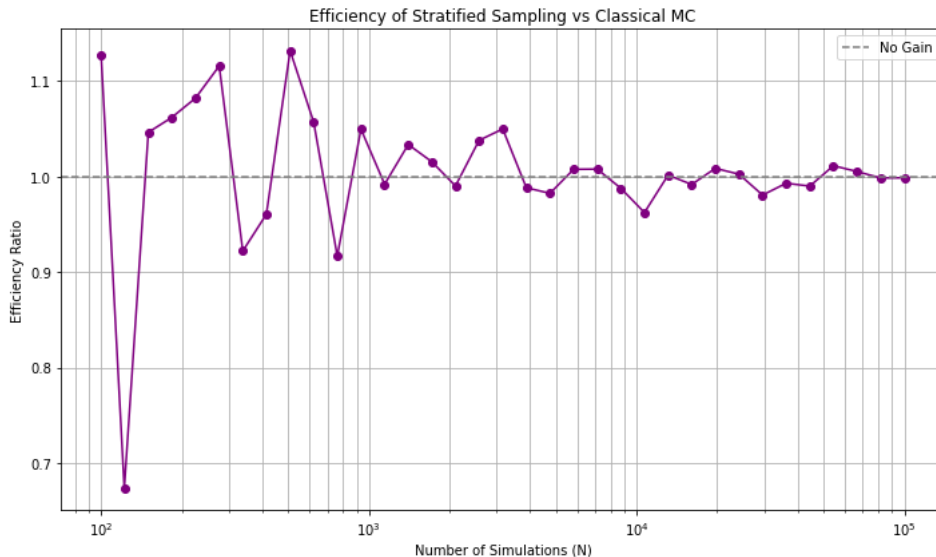


Figure 8: Comparison efficiency between Stratified Sampling and MC simulations

CHAPTER 4: ALTERNATIVES AND EXTENSIONS

4.1 BINOMIAL TREES

The Black Sholes method takes the world from a continuous perspective. Another way to see it is a series of time steps, which is actually closer to the truth when we look at how financial instruments are traded and quoted on the market by computers. The binomial tree model, far from being outdated, thus still makes sense in today's world.

This model, called Cox-Ross-Rubinstein's model, relies on the assumption that there is only one risky and one risk-free asset trade, trading is opened at discrete times, no transaction costs, constant implied volatility, two possible outcomes (up and down) and a duration of length δt .

To get to the formula for the one-step scenario of a call option, we need to place ourselves in a risk-neutral world again, where the payoff of the call is equal to the value of a portfolio consisting of Δ shares and an amount B of cash invested in a risk-free asset: $c_T = \Delta S_T + B e^{rT}$ (4.1)

The two possible paths are (u(resp. d) denoting the upward (resp.downward) factor):

$$1) \Delta S_u + B e^{rT} = c_u$$

$$2) \Delta S_d + B e^{rT} = c_d$$

Giving us the replication portfolio which value at the beginning is equal to the price of the option:

$$\Delta = \frac{c_u - c_d}{S(u-d)} \quad B = -e^{-rT} \frac{dc_u - uc_d}{(u-d)}$$

If we develop equation (4.1) using the above formulas and denote $p = \frac{e^{rT} - d}{u - d}$ we get:

$$c = e^{-rT} [p c_u + (1-p) c_d]$$

Which, if we interpret p as a measure of probability of the option today is the expected future payoff discounted at the risk-free rate:

$$c = e^{-rT} E_p[c_T]$$

If we want to use a multiperiod model, all we have to do then is to start from the end of the tree and bootstrap the whole tree until the root is reached.

However, we are still missing a key element to be able to price options using this technique: the values taken by the u and d parameters. They have to be chosen so that the return and variance outlined by the tree match the ones of the real world: $e^{r\delta t} = pu + (1 - p)d$ and $\sigma^2\delta t = pu^2 + (1 - p)d^2 - e^{2r\delta t}$

As we can see we have two equations with three unknown variables, thus leading to an infinity of possible calibrations.

A possible and widely used calibration is the one of Cox-Ross-Rubinstein which uses:

$$u_{CRR} = e^{\sigma\sqrt{\delta t}} \quad \text{and} \quad d_{CRR} = 1/u_{CRR}$$

Ex: if we use this on the first AAPL stock we studied, it needs only about 90 steps to stabilize around the price from the Black Sholes model, far from the hundreds of thousand simulations required for Monte Carlo.

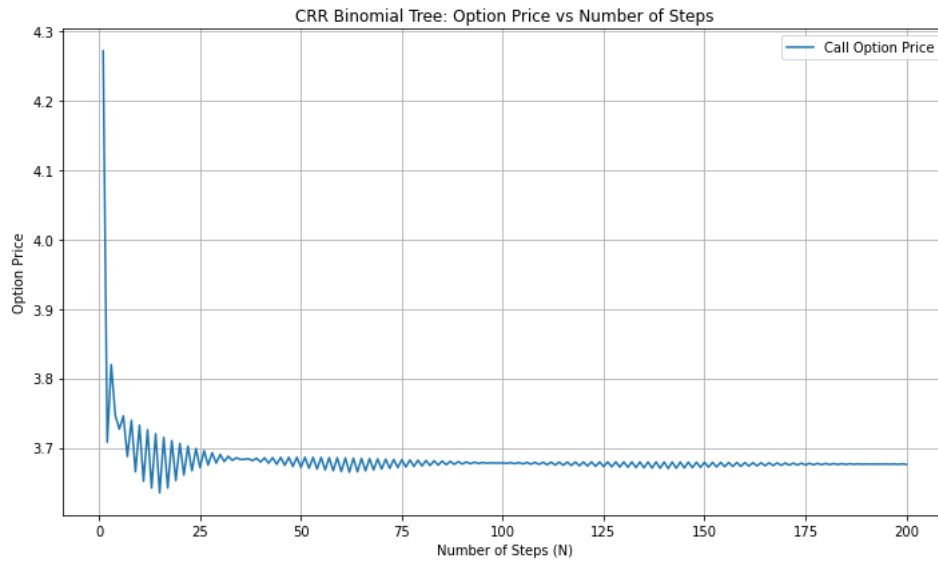


Figure 9: evolution of the price with regard to the number of steps used

This technique gives good results for a simple European call but is also the main one used to price American options as its multi-period model perfectly fits the needs for this type of option, making it the go to method when pricing them.

4.2 LOCAL VOLATILITY MODELS

Local volatility models assume that volatility is a deterministic function of time and the asset price, which translates to: $\sigma = \sigma(t, S_t)$

This is a major change from Black Sholes as here volatility is not constant but depends on two parameters.

The intuition behind this choice is that implied volatilities differ for different strikes and maturities (this is called the volatility smile/skew) and local volatility tries to fit this entire implied volatility surface at a given moment.

In 1994, Dupire developed a model providing a formula to express the volatility:

$$\sigma_{loc}^2(t, K) = \frac{\frac{\partial C}{\partial t} + rK \frac{\partial C}{\partial K}}{\frac{1}{2} \frac{\partial^2 C}{\partial K^2}}$$

Where $C(t, K)$ is the price of the call option as a function of time and strike.

Then this formula can be used to build a modified version of the Black Sholes PDE:

$$\frac{\partial C}{\partial t} + \frac{1}{2} \sigma_{loc}^2(t, S) S^2 \frac{\partial^2 C}{\partial S^2} + rS \frac{\partial C}{\partial S} - rC = 0$$

Ultimately, this equation is solved numerically to find the price of the option, for example by using finite differences.

This model is often used to price Vanilla options as it isn't fit for path-dependent options. For them, stochastic models can be used but we will not talk about them.

CONCLUSION

Throughout this thesis, we constructed a progression from the fundamental definitions of options to advanced numerical methods for their pricing. Beginning with foundational concepts allowed us to establish the necessary framework for the analysis that followed. By focusing on the Black-Scholes model, we acknowledged its central role in quantitative finance while also recognizing its limitations in practical applications. This motivated our exploration of Monte Carlo simulations, where we tested their accuracy and efficiency. A significant insight was gained from implementing variance reduction techniques, which notably enhanced computational performance and deepened our understanding of stochastic simulation. Finally, our examination of alternative methods with binomial trees and local volatility models highlighted the variety of approaches available and the trade-offs between model complexity and market realism.

Overall, this work underscores the importance of combining analytical insight with numerical methods in financial modeling. Each method explored offers its own advantages and limitations, and the choice of model must be carefully tailored to the financial product and market conditions in question. Future work may involve further exploration of stochastic volatility models, machine learning techniques for volatility surface estimation, or calibration methods for exotic derivatives, all of which remain active areas of research in financial engineering.

BIBLIOGRAPHY

1. Hull, John C. *Options, Futures, and Other Derivatives*. 10th ed. Boston: Pearson, 2017
2. Bouzoubaa, Mohamed, and Adel Osseiran. *Exotic Options and Hybrids: A Guide to Structuring, Pricing and Trading*. Hoboken, NJ: John Wiley & Sons, 2010
3. Laurent Deville, *Introduction to Derivatives*, EDHEC Business School
4. Nicole El Karoui, *Couverture des risques dans les marchés financiers*, https://math.uni.lu/thalmaier/finance/el_karoui.pdf
5. https://en.wikipedia.org/wiki/Monte_Carlo_methods_for_option_pricing
6. Sigman, Karl. *Antithetic Variates*. Columbia University, n.d. <http://www.columbia.edu/~ks20/4404-Sigman/4404-Notes-ATV.pdf>.
7. <https://www.barchart.com/stocks/quotes/AAPL>
8. Goodman, David. *Week 12: Monte Carlo Methods*. Lecture notes, New York University, 2020. <https://math.nyu.edu/~goodman/teaching/MonteCarlo20/week12/Week12.pdf>
9. Jeremy Tejada, *Stratified Sampling in Monte Carlo Simulation: Motivation, Design, and Sampling Error*. https://jjtejada.wordpress.com/wpcontent/uploads/2014/01/stratified_sampling.pdf

APPENDIX

June 16, 2025

```
[ ]: import math
import time
import matplotlib.pyplot as plt
from scipy.stats import norm
import numpy as np
import yfinance as yf
from scipy.special import erfinv
```

```
[ ]: #Black Sholes Model

# Parameters of AAPL, information from Barchart.com for a european call
#   ↪expiring in 4 days
#consulted on 02/06/2025

tps1=time.time()
S = 201.70
K = 200
T = 4 / 365
sigma = 0.3202
r = 0.0412

# d1 and d2 computations
d1 = (math.log(S / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * math.sqrt(T))
d2 = d1 - sigma * math.sqrt(T)

# BS formula with dividends
call_bs_price = S * norm.cdf(d1) - K * math.exp(-r * T) * norm.cdf(d2)
print(f"Call price : {call_bs_price:.3f}")
tps2= time.time()
print(tps2-tps1,"s")
```

```
[ ]: #Monte Carlo 1

tps3=time.time()
# Parameters
S0 = 201.70      # Spot price
K = 200          # Strike
T = 4/365        # Time to maturity (in years)
```



```

r = 0.0412          # Risk-free rate
sigma = 0.3202      # Volatility
N = 10000000        # Number of simulations
delta_T=1/4

# Generate standard normal random variables
Z = np.random.randn(N)

# Simulate asset price at maturity under risk-neutral measure
ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z)

# Compute discounted payoff
payoffs = np.exp(-r * T) * np.maximum(ST - K, 0)

# Monte Carlo price
call_MC_price = np.mean(payoffs)

print(f"Estimated Call Price: {call_MC_price:.4f}")
tps4= time.time()
print(tps4-tps3,"s")

#on average 0.24s

```

```

[ ]: #Convergence of MC

# Simulation counts (log scale)
simulation_counts = np.logspace(2, 6, num=50, dtype=int)
estimated_prices = []

# Monte Carlo simulations
for N in simulation_counts:
    Z = np.random.randn(N)
    ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z)
    payoffs = np.exp(-r * T) * np.maximum(ST - K, 0)
    estimated_prices.append(np.mean(payoffs))

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(simulation_counts, estimated_prices, label='Monte Carlo Estimate',
        marker='o', linestyle='--')
plt.axhline(call_bs_price, color='red', linestyle='-', label='Black-Scholes
        Price')
plt.xscale('log')
plt.xlabel('Number of Simulations (log scale)')

```

```

plt.ylabel('Estimated Call Price')
plt.title('Monte Carlo Convergence for AAPL European Call Option')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

```

[ ]: # Use of Central Limit Theorem to get number of simulations required to get the
      ↳desired bandwidth

# Step 1: Pilot simulation on small nb of simulations to get an idea of the
      ↳value of the standard deviation
NO = 100000
Z = np.random.randn(NO)
ST = S0 * np.exp((r - q - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z)
payoffs = np.exp(-r * T) * np.maximum(ST - K, 0)
s_hat = np.std(payoffs, ddof=1)

# Step 2: Solve for required N
desired_half_width = 0.005
z = 1.96
N_required = int(np.ceil((z * s_hat / desired_half_width) ** 2))

print(f"Estimated std dev: {s_hat:.4f}")
print(f"Required simulations for 95% CI < 0.01: {N_required}")

```

```

[ ]: #Monte Carlo using Multiple Steps, useful for path dependent options

```

```

tps5= time.time()

N = 1_000_000      # Number of simulations
steps = 20          # Number of time steps, here 20 over 4 days

dt = T / steps      # Time increment per step
discount_factor = np.exp(-r * T)

# Simulate paths
Z = np.random.randn(N, steps) # Shape: (N, steps)
increments = (r - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z
log_S_paths = np.cumsum(increments, axis=1)
S_paths = S0 * np.exp(log_S_paths)

# Get the final price at maturity
ST = S_paths[:, -1]

# Compute discounted payoff

```

```

payoffs = discount_factor * np.maximum(ST - K, 0)

# Monte Carlo estimate
call_price = np.mean(payoffs)

print(f"Estimated Call Price (Multi-Step): {call_price:.4f}")
tps6= time.time()
print(tps6-tps5,"s")

#on average 0.46s

```

```

[ ]: #Down and out call
import numpy as np
tps_DOU1= time.time()
# Parameters
S0 = 201.70      # Spot price
K = 200          # Strike
B = 190          # Barrier level (down-and-out)
T = 4/365        # Time to maturity
r = 0.0412       # Risk-free rate
sigma = 0.3202   # Volatility
N = 1000000      # Number of simulations
steps = 20       # Nb of steps

dt = T / steps
discount_factor = np.exp(-r * T)

# Simulate paths
Z = np.random.randn(N, steps)
S_paths = S0 * np.exp(np.cumsum(((r - q - 0.5 * sigma**2) * dt + sigma * np.
    ↪sqrt(dt) * Z), axis=1))

# Insert initial spot
S_paths = np.insert(S_paths, 0, S0, axis=1)

# Check for barrier breach
barrier_breached = np.any(S_paths <= B, axis=1)

# Terminal prices
ST = S_paths[:, -1]
payoffs = np.where(barrier_breached, 0, np.maximum(ST - K, 0))

# Price
price = discount_factor * np.mean(payoffs)
print(f"Down-and-Out Call Price: {price:.4f}")

tps_DOU2= time.time()

```

```
print(tps_DOU2-tps_DOU1,"s")
```

```
[ ]: #Asian Option
```

```
# Same parameters as above
tps_as1= time.time()
# Simulate asset paths
Z = np.random.randn(N, steps)
S_paths = S0 * np.exp(np.cumsum(((r - 0.5 * sigma**2) * dt + sigma * np.
    ↪sqrt(dt) * Z), axis=1))

# Insert initial price
S_paths = np.insert(S_paths, 0, S0, axis=1)

# Compute arithmetic average
S_avg = np.mean(S_paths, axis=1)

# Asian option payoff
payoffs = np.maximum(S_avg - K, 0)

# Price
asian_price = discount_factor * np.mean(payoffs)
print(f"Asian Call Price (Arithmetic Average): {asian_price:.4f}")

tps_as2= time.time()
print(tps_as2-tps_as1,"s")
```

```
[ ]: #Code to show the price of a Down-and-Out Call Option when we move the barrier ↪
    ↪Level
```

```
# Function to compute Monte Carlo price of down-and-out call
def monte_carlo_down_and_out(S0, K, B, T, r, sigma, N, steps):
    if B >= S0:
        return 0.0 # All paths are knocked out at start

    dt = T / steps
    Z = np.random.randn(N, steps)
    drift = (r - 0.5 * sigma**2) * dt
    diffusion = sigma * np.sqrt(dt) * Z
    log_returns = drift + diffusion
    log_paths = np.cumsum(log_returns, axis=1)
    S_paths = S0 * np.exp(log_paths)
    S_paths = np.insert(S_paths, 0, S0, axis=1)

    barrier_breached = np.any(S_paths <= B, axis=1)
```

```

ST = S_paths[:, -1]
payoffs = np.where(barrier_breached, 0, np.maximum(ST - K, 0))

return discount_factor * np.mean(payoffs)

# Valid barrier levels strictly below the spot price
barrier_levels = np.linspace(180, 203, 50)
prices = []

# Run simulation for each barrier level
for B in barrier_levels:
    price = monte_carlo_down_and_out(S0, K, B, T, r, sigma, N, steps)
    prices.append(price)

# Plot the results
plt.figure(figsize=(10, 6))
plt.plot(barrier_levels, prices, label="Down-and-Out Call (MC)",
         color='darkgreen')
plt.xlabel("Barrier Level (B)")
plt.ylabel("Option Price")
plt.title("Down-and-Out Call Option Price vs Barrier Level (Monte Carlo)")
plt.grid(True)
plt.legend()
plt.show()

```

```

[ ]: #Worst-Of option

#Correlation Matrix calculations using yahoo finance

tickers = ['AAPL', 'NVDA']
start_date = '2024-06-01'
end_date = '2025-06-01'

data = yf.download(tickers, start=start_date, end=end_date, progress=False,
                  auto_adjust=True)
adj_close = data['Close']
returns = adj_close.pct_change().dropna()
correlation = returns.corr().loc['AAPL', 'NVDA']

print(f"Correlation between AAPL and NVDA (daily returns): {correlation:.4f}")

tickers2 = ['AAPL', 'TSLA']
data2 = yf.download(tickers2, start=start_date, end=end_date, progress=False,
                  auto_adjust=True)
adj_close2 = data2['Close']

```

```

returns2 = adj_close2.pct_change().dropna()
correlation2 = returns2.corr().loc['AAPL', 'TSLA']

print(f"Correlation between AAPL and NVDA (daily returns): {correlation2:.4f}")

tickers3 = ['TSLA', 'NVDA']
data3 = yf.download(tickers3, start=start_date, end=end_date, progress=False,
    ↪auto_adjust=True)
adj_close3 = data3['Close']
returns3 = adj_close3.pct_change().dropna()
correlation3 = returns3.corr().loc['TSLA', 'NVDA']

print(f"Correlation between TSLA and NVDA (daily returns): {correlation3:.4f}")

```

```

[ ]: # Parameters for Worst-of option
S0 = np.array([201.70, 135.50, 343.83]) # Initial prices
sigma = np.array([0.3202, 0.36, 0.64]) # Volatilities

rho = np.array([[1.0, 0.44, 0.50],
                [0.44, 1.0, 0.50],
                [0.50, 0.50, 1.0]]) # Correlation matrix

T = 4/365 # Time to maturity (in years)
r = 0.0412 # Risk-free rate
K = 0.05 # Performance-based strike (% rather than price)

N = 100000 # Simulations
n_assets = len(S0)

# Cholesky decomposition for correlated normals
L = np.linalg.cholesky(rho)
Z = np.random.randn(N, n_assets)
correlated_Z = Z @ L.T

# Simulate terminal prices
drift = (r - 0.5 * sigma**2) * T
diffusion = sigma * np.sqrt(T) * correlated_Z
log_returns = drift + diffusion
S_T = S0 * np.exp(log_returns)

# Compute performance of each asset
perf = (S_T - S0) / S0

# Worst performance across assets
worst_perf = np.min(perf, axis=1)

```

```

# Payoff
payoffs = np.maximum(worst_perf - K, 0)
price = np.exp(-r * T) * np.mean(payoffs)

print(f"Worst-of Option Price (Performance-based): {price:.4f}")

```

[]: *#1st Monte Carlo Improvement:*

```

# Antithetic variates

option_type = "call"

N_values = np.logspace(2, 5, num=70, dtype=int)

classic_prices = []
antithetic_prices = []

# === Simulation ===
for N in N_values:
    # Classic Monte Carlo
    Z = np.random.randn(N)
    ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z)

    if option_type == "call":
        payoff = np.maximum(ST - K, 0)
    else:
        payoff = np.maximum(K - ST, 0)

    price_classic = np.exp(-r * T) * np.mean(payoff)
    classic_prices.append(price_classic)

    # Antithetic method
    Z_half = Z[:N//2] # use the same base for better comparison
    Z_antithetic = -Z_half

    ST1 = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z_half)
    ST2 = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) *
    ↪Z_antithetic)

    if option_type == "call":
        payoff1 = np.maximum(ST1 - K, 0)
        payoff2 = np.maximum(ST2 - K, 0)
    else:
        payoff1 = np.maximum(K - ST1, 0)
        payoff2 = np.maximum(K - ST2, 0)

```

```

    payoff_avg = 0.5 * (payoff1 + payoff2)
    price_antithetic = np.exp(-r * T) * np.mean(payoff_avg)
    antithetic_prices.append(price_antithetic)

true_price_estimate = antithetic_prices[-1]

classic_std = np.std(payoff)          # From classic MC
antithetic_std = np.std(payoff_avg)   # From antithetic MC

efficiency = (classic_std ** 2) / (antithetic_std ** 2)
print(f"Relative Efficiency: {efficiency:.2f}x")

#Comparing convergence visually

plt.figure(figsize=(10, 6))
plt.plot(N_values, classic_prices, label="Classic Monte Carlo", linestyle='--',
        color="blue")
plt.plot(N_values, antithetic_prices, label="Antithetic method", color="green")
plt.axhline(true_price_estimate, color="gray", linestyle=":", label=f"limit_
        price {true_price_estimate:.4f}")
plt.xscale("log")
plt.xlabel("number of simulations")
plt.ylabel("estimated price of the option")
plt.title(f"Price convergence (European {option_type.capitalize()})")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

```

[ ]: #2nd Monte Carlo Improvement:

# Control variates

# Payoff function
def call_payoff(ST, K):
    return np.maximum(ST - K, 0)

E_ST = S0 * np.exp(r * T)

# Simulation settings
N_values = np.logspace(2, 5, num=35, dtype=int)
price_classic, price_control = [], []
error_std_classic, error_std_control = [], []

for N in N_values:

```



```

Z = np.random.randn(N)
ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z)

# Classical Monte Carlo
payoff = call_payoff(ST, K)
disc_payoff = np.exp(-r * T) * payoff
price_classic.append(np.mean(disc_payoff))
error_std_classic.append(np.std(disc_payoff, ddof=1) / np.sqrt(N))

# Control variate
cov = np.cov(disc_payoff, ST)[0, 1]
beta = cov / np.var(ST)
adj_payoff = disc_payoff - beta * (ST - E_ST)
price_control.append(np.mean(adj_payoff))
error_std_control.append(np.std(adj_payoff, ddof=1) / np.sqrt(N))

# We take the last element to get the most relevant data
classic_var = error_std_classic[-1]**2
control_var = error_std_control[-1]**2
efficiency = classic_var / control_var

print(f"Relative Efficiency (Control Variate vs Classical MC): {efficiency:.2f}x")

# Plot estimated prices
plt.figure(figsize=(10, 6))
plt.plot(N_values, price_classic, '--', label="Classical MC", color="blue")
plt.plot(N_values, price_control, label="Control Variate", color="green")
plt.axhline(y=price_control[-1], color='gray', linestyle=':', label="Stable Estimate")
plt.xscale("log")
plt.xlabel("Number of Simulations (N)")
plt.ylabel("Estimated Option Price")
plt.title("Convergence of Option Price Estimators")
plt.legend()
plt.grid(True)
plt.show()

# Plot standard errors
plt.figure(figsize=(10, 6))
plt.plot(N_values, error_std_classic, '--', label="Std Error (Classical)", color="blue")
plt.plot(N_values, error_std_control, label="Std Error (Control Variate)", color="green")

```

```

plt.xscale("log")
plt.yscale("log")
plt.xlabel("Number of Simulations (N)")
plt.ylabel("Standard Error")
plt.title("Standard Error Comparison")
plt.legend()
plt.grid(True, which="both")
plt.show()

```

[]: *#3rd Monte Carlo Improvement:*

```

# Stratified Sampling

# === Classical Monte Carlo ===
def monte_carlo_standard(N):
    Z = np.random.randn(N)
    ST = S0 * np.exp((r - 0.5 * sigma**2)*T + sigma * np.sqrt(T) * Z)
    payoff = call_payoff(ST, K)
    disc_payoff = np.exp(-r * T) * payoff
    return np.mean(disc_payoff), np.std(disc_payoff, ddof=1) / np.sqrt(N)

# === Stratified Sampling ===
def monte_carlo_stratified(N):
    U = (np.arange(N) + np.random.uniform(0, 1, N)) / N # Uniform stratified
    Z = np.sqrt(2) * erfinv(2 * U - 1) # Inverse transform of normal
    ST = S0 * np.exp((r - 0.5 * sigma**2)*T + sigma * np.sqrt(T) * Z)
    payoff = call_payoff(ST, K)
    disc_payoff = np.exp(-r * T) * payoff
    return np.mean(disc_payoff), np.std(disc_payoff, ddof=1) / np.sqrt(N)

from scipy.special import erfinv

N_values = np.logspace(2, 5, 35, dtype=int)
price_std, price_strat = [], []
std_err_std, std_err_strat = [], []

# === Simulations ===
for N in N_values:
    p_std, err_std = monte_carlo_standard(N)
    p_strat, err_strat = monte_carlo_stratified(N)

    price_std.append(p_std)
    std_err_std.append(err_std)
    price_strat.append(p_strat)
    std_err_strat.append(err_strat)

```

```

efficiency_ratios = (np.array(std_err_std) / np.array(std_err_strat))**2

# === Plot Efficiency Ratio ===
plt.figure(figsize=(10, 6))
plt.plot(N_values, efficiency_ratios, marker='o', color='purple')
plt.axhline(y=1, color="gray", linestyle="--", label="No Gain")
plt.xscale("log")
plt.xlabel("Number of Simulations (N)")
plt.ylabel("Efficiency Ratio")
plt.title("Efficiency of Stratified Sampling vs Classical MC")
plt.legend()
plt.grid(True, which="both")
plt.tight_layout()
plt.show()

# Plot estimated prices
plt.figure(figsize=(10, 6))
plt.plot(N_values, price_std, '--', label="Classical Monte Carlo", color="red")
plt.plot(N_values, price_strat, label="Stratified Sampling", color="green")
plt.axhline(price_strat[-1], linestyle=":", color="gray", label="Stable_
↳ Estimate")
plt.xscale("log")
plt.xlabel("Number of Simulations (N)")
plt.ylabel("Estimated Price")
plt.title("Convergence of Estimated Option Price\n(Monte Carlo vs Stratified_
↳ Sampling)")
plt.legend()
plt.grid(True)
plt.show()

# Plot standard errors
plt.figure(figsize=(10, 6))
plt.plot(N_values, std_err_std, '--', label="Std Error (Classical)",
↳ color="red")
plt.plot(N_values, std_err_strat, label="Std Error (Stratified)", color="green")
plt.xscale("log")
plt.yscale("log")
plt.xlabel("Number of Simulations (N)")
plt.ylabel("Standard Error")
plt.title("Comparison of Standard Errors")
plt.legend()
plt.grid(True, which="both")
plt.show()

```

```
[ ]: #Binomial tree model
```

```

#Pricing of a European Call or Put using Cox-Ross-Rubinstein's model in
↳relation to the number of steps

tpsCRR1=time.time()
def crr_option_price(S0, K, T, r, sigma, N, option_type="call"):
    dt = T / N
    u = np.exp(sigma * np.sqrt(dt))
    d = 1 / u
    p = (np.exp(r * dt) - d) / (u - d)
    discount = np.exp(-r * dt)

    ST = np.array([S0 * (u ** j) * (d ** (N - j)) for j in range(N + 1)])

    if option_type == "call":
        option_values = np.maximum(ST - K, 0)
    elif option_type == "put":
        option_values = np.maximum(K - ST, 0)
    else:
        raise ValueError("option_type must be 'call' or 'put'")

    for i in range(N - 1, -1, -1):
        option_values = discount * (p * option_values[1:i + 2] + (1 - p) *
↳option_values[0:i + 1])

    return option_values[0]

# Parameters
S0 = 201.70
K = 200
T = 4/365
sigma = 0.3202
r = 0.0412
N=10
option_type = "call" # "call" or "put"

# Range of steps to test
steps_range = range(1, 201)
prices = [crr_option_price(S0, K, T, r, sigma, N, option_type) for N in
↳steps_range]

tpsCRR2= time.time()
print(tpsCRR2-tpsCRR1,"s")
print(prices)

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(steps_range, prices, label=f"{option_type.capitalize()} Option Price")

```

```
plt.xlabel("Number of Steps (N)")
plt.ylabel("Option Price")
plt.title("CRR Binomial Tree: Option Price vs Number of Steps")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```